Fix-Hub: Complete UML Diagrams Documentation

> **Project:** Fix-Hub Car Maintenance Management System
> **Document Version:** 1.0
> **Last Updated:** December 22, 2025
> **Coverage:** Complete UML diagrams for all roles and features

Table of Contents

4.5.1 Use Case Tables

Customer Use Cases

UC-C01: Register as Customer

Field	Description
Use Case ID	UC-C01
Use Case Name	Register as Customer
Primary Actor	Customer
Preconditions	User has email and phone number
Postconditions	Customer account is created in Firestore
Trigger	User selects "Register as Customer"
Main Flow	1. User enters email, password, name, and phone 2. System validates input format 3. System creates Firebase Auth account 4. System creates Firestore user profile with role=customer 5. System logs user in automatically 6. System redirects to Customer Dashboard
Alternative Flow	3a. If email already exists, show error 3b. If password too weak, show error 4a. If network error, retry with backoff

UC-C02: Book Service Maintenance

Field	Description

Field	Description
Use Case ID	UC-C02
Use Case Name	Book Service Maintenance
Primary Actor	Customer
Preconditions	Customer is logged in and has added at least one car
Postconditions	Booking created with status=pending, visible to all technicians
Trigger	Customer clicks "Book Service"
Main Flow	1. Customer selects car from their list 2. Customer selects maintenance type (regular/repair/inspection/emergency) 3. Customer browses service catalog and adds items 4. Customer selects date and time slot 5. System checks slot availability 6. Customer enters description/notes 7. Customer applies discount code (optional) 8. System validates offer code if provided 9. Customer confirms booking 10. System saves booking to Firestore 11. System sends notifications to all technicians 12. System shows confirmation to customer
Alternative Flow	5a. If slot unavailable, show alternative slots 8a. If offer code invalid/expired, show error 10a. If network error, retry

UC-C03: Track Maintenance Status

Field	Description
Use Case ID	UC-C03
Use Case Name	Track Maintenance Status
Primary Actor	Customer
Preconditions	Customer has active bookings
Postconditions	Customer views current status
Trigger	Customer opens "My Bookings"
Main Flow	1. System queries bookings where userId=currentUser 2. System displays bookings with real-time status 3. Customer clicks on booking to see details 4. System shows: assigned technician, service items, current status, estimated completion

Field	Description
Alternative Flow	2a. If no bookings exist, show "No bookings yet"

UC-C04: Rate Completed Service

Field	Description
Use Case ID	UC-C04
Use Case Name	Rate Completed Service
Primary Actor	Customer
Preconditions	Booking status=completed and isPaid=true and rating is null
Postconditions	Booking updated with rating and comment
Trigger	Customer clicks "Rate Service" on completed booking
Main Flow	1. System shows rating dialog 2. Customer selects stars (1-5) 3. Customer enters comment (optional) 4. Customer submits rating 5. System updates booking with rating, ratingComment, ratedAt 6. System sends notification to assigned technician
Alternative Flow	2a. If already rated, show "Already rated"

UC-C05: Download Invoice PDF

Field	Description
Use Case ID	UC-C05
Use Case Name	Download Invoice PDF
Primary Actor	Customer
Preconditions	Booking isPaid=true
Postconditions	PDF invoice downloaded to device
Trigger	Customer clicks "Download Invoice"
Main Flow	1. System retrieves booking details 2. System generates PDF with: customer info, car details, service items, labor cost, discount, tax, total 3. System downloads PDF to device 4. System shows success message

Field	Description
Alternative Flow	2a. If PDF generation fails, show error

UC-C06: Chat with AI Assistant

Field	Description
Use Case ID	UC-C06
Use Case Name	Chat with AI Assistant
Primary Actor	Customer
Preconditions	Customer is logged in
Postconditions	Conversation saved to Firestore
Trigger	Customer opens Chatbot
Main Flow	1. System loads previous conversation history 2. Customer types message 3. System sends message to Gemini AI 4. System receives AI response 5. System displays response to customer 6. System saves conversation to Firestore
Alternative Flow	3a. If API error, show "Unable to connect"

Technician Use Cases

UC-T01: View Available Jobs

Field	Description
Use Case ID	UC-T01
Use Case Name	View Available Jobs
Primary Actor	Technician
Preconditions	Technician is logged in
Postconditions	List of pending bookings displayed
Trigger	Technician opens dashboard
Main Flow	1. System queries bookings where status=pending 2. System displays job list with: customer name, car details, maintenance type, scheduled date 3. Technician views job details

Field	Description
Alternative Flow	1a. If no jobs available, show "No jobs available"

UC-T02: Pick and Start Job

Field	Description
Use Case ID	UC-T02
Use Case Name	Pick and Start Job
Primary Actor	Technician
Preconditions	Job status=pending
Postconditions	Job status=inProgress, technician assigned
Trigger	Technician clicks "Start Job"
Main Flow	1. System updates booking: status=inProgress, assignedTechnicians=[techId], startedAt=now 2. System sends notification to customer 3. System redirects technician to job details page
Alternative Flow	1a. If job already taken by another technician, show error

UC-T03: Add Service Items and Parts

Field	Description
Use Case ID	UC-T03
Use Case Name	Add Service Items and Parts
Primary Actor	Technician
Preconditions	Technician is working on job (status=inProgress)
Postconditions	Service items added to booking, inventory updated
Trigger	Technician clicks "Add Items"
Main Flow	1. Technician searches for parts/services 2. Technician selects item and enters quantity 3. System checks inventory availability 4. Technician adds item to booking 5. System creates inventory_transaction (type=out) 6. System decrements inventory.currentStock 7. System checks if currentStock < threshold 8. If yes, create low_stock_alert and notify admin

Field	Description
Alternative Flow	3a. If insufficient stock, show error

UC-T04: Complete Job

Field	Description
Use Case ID	UC-T04
Use Case Name	Complete Job
Primary Actor	Technician
Preconditions	Job status=inProgress
Postconditions	Job status=completedPendingPayment
Trigger	Technician clicks "Complete Job"
Main Flow	1. Technician enters completion notes 2. System updates booking: status=completedPendingPayment, completedAt=now 3. System calculates totalCost (parts + labor + tax - discount) 4. System sends notification to cashier 5. System sends notification to customer
Alternative Flow	None

Admin Use Cases

UC-A01: Generate Invite Code

Field	Description
Use Case ID	UC-A01
Use Case Name	Generate Invite Code
Primary Actor	Admin
Preconditions	Admin is logged in
Postconditions	New invite code created in Firestore
Trigger	Admin clicks "Generate Code"
Main Flow	1. Admin selects role (technician/admin/cashier) 2. Admin enters max uses 3. System generates unique 8-character code 4. System creates invite_code document with: code, role, maxUses, usedCount=0, isActive=true, createdBy=adminId 5. System displays code to admin with copy button

Field	Description
Alternative Flow	4a. If code generation fails, retry

UC-A02: Manage Users

Field	Description
Use Case ID	UC-A02
Use Case Name	Manage Users
Primary Actor	Admin
Preconditions	Admin is logged in
Postconditions	User account activated/deactivated
Trigger	Admin opens "Users Management"
Main Flow	1. System displays all users with: name, email, role, isActive status 2. Admin selects user 3. Admin toggles isActive status 4. System updates user document
Alternative Flow	None

UC-A03: Approve/Reject Refund

Field	Description
Use Case ID	UC-A03
Use Case Name	Approve or Reject Refund
Primary Actor	Admin
Preconditions	Refund status=requested
Postconditions	Refund status=approved or rejected
Trigger	Admin opens "Refund Requests"
Main Flow	1. System displays refund requests with status=requested 2. Admin views refund details: booking info, refund reason, amount 3. Admin reviews booking history 4. Admin clicks "Approve" or "Reject" 5. System updates refund: status=approved/rejected, approvedBy=adminId, approvedAt=now 6. System sends notification to cashier 7. If rejected, system sends notification to customer

Field	Description
Alternative Flow	None

UC-A04: Create Promotional Offer

Field	Description
Use Case ID	UC-A04
Use Case Name	Create Promotional Offer
Primary Actor	Admin
Preconditions	Admin is logged in
Postconditions	New offer created, customers notified
Trigger	Admin clicks "Create Offer"
Main Flow	1. Admin enters: title, description, discountPercentage, startDate, endDate 2. System generates unique offer code 3. Admin uploads offer image (optional) 4. System creates offer document with: code, type=discount, isActive=true, createdBy=adminId 5. System queries all users where role=customer 6. System sends notification to all customers 7. System displays success message
Alternative Flow	2a. If code generation fails, retry

UC-A05: View System Reports

Field	Description
Use Case ID	UC-A05
Use Case Name	View System Reports
Primary Actor	Admin
Preconditions	Admin is logged in
Postconditions	Reports displayed
Trigger	Admin opens "Reports"

Field	Description
Main Flow	1. Admin selects report type (profit/performance/bookings) 2. Admin selects date range 3. System aggregates data from Firestore 4. System calculates: total revenue, completed bookings, active technicians, average rating 5. System displays charts and statistics
Alternative Flow	3a. If no data available, show "No data"

Cashier Use Cases

UC-CS01: Process Payment

Field	Description
Use Case ID	UC-CS01
Use Case Name	Process Payment
Primary Actor	Cashier
Preconditions	Booking status=completedPendingPayment
Postconditions	Booking isPaid=true, status=completed, invoice created
Trigger	Cashier opens "Pending Payments"
Main Flow	1. System displays bookings with status=completedPendingPayment 2. Cashier selects booking 3. System displays cost breakdown: subtotal, discount, tax, total 4. Cashier selects payment method (cash/card/digital) 5. Cashier processes payment 6. System updates booking: isPaid=true, paidAt=now, cashierId, paymentMethod, status=completed 7. System creates invoice document 8. System sends invoice to customer 9. System sends "payment successful" notification to customer
Alternative Flow	5a. If payment fails, show error and retry

UC-CS02: Initiate Refund Request

Field	Description
Use Case ID	UC-CS02

Field	Description
Use Case Name	Initiate Refund Request
Primary Actor	Cashier
Preconditions	Booking isPaid=true
Postconditions	Refund created with status=requested
Trigger	Customer requests refund at cashier
Main Flow	1. Cashier retrieves booking details 2. Cashier enters refund reason and amount 3. Cashier adds customer notes (optional) 4. System creates refund document: bookingId, refundAmount, reason, status=requested, requestedBy=cashierId, requestedAt=now 5. System sends notification to admin for approval
Alternative Flow	1a. If booking not paid, show error

UC-CS03: View Profit Reports

Field	Description
Use Case ID	UC-CS03
Use Case Name	View Profit Reports
Primary Actor	Cashier
Preconditions	Cashier is logged in
Postconditions	Profit data displayed
Trigger	Cashier opens "Reports"
Main Flow	1. Cashier selects date range 2. System queries bookings where isPaid=true within date range 3. System sums totalCost from all paid bookings 4. System displays total revenue and transaction count
Alternative Flow	2a. If no transactions, show "No data"

4.6 Activity Diagrams

4.6.1 Customer Activity Diagram

Complete customer journey from registration through service booking, payment, and rating.

```
flowchart TD
    Start([Customer Opens App]) --> CheckAuth{Has Account?}
    CheckAuth -->|No| Register[Register as Customer]
    Register --> EnterDetails[Enter Email Password Name Phone]
    EnterDetails --> CreateAuth[System Creates Firebase Auth]
    CreateAuth --> CreateProfile[System Creates Firestore Profile]
    CreateProfile --> Login

    CheckAuth -->|Yes| Login[Login]
    Login --> Dashboard[Customer Dashboard]

    Dashboard --> Action{Choose Action}

    Action -->|Book Service| HasCar{Has Car?}
    HasCar -->|No| AddCar[Add Car]
    HasCar -->|Yes| SelectCar[Select Car]
    AddCar --> SelectCar
    SelectCar --> SelectType[Select Maintenance Type]
    SelectType --> SelectDateTime[Choose Date Time]
    SelectDateTime --> EnterNotes[Enter Description]
    EnterNotes --> ApplyOffer{Have Discount Code?}
    ApplyOffer -->|Yes| EnterCode[Enter Code]
    EnterCode --> ApplyDiscount[Apply Discount]
    ApplyOffer -->|No| Confirm
    ApplyDiscount --> Confirm[Confirm Booking]
    Confirm --> Dashboard

    Action -->|Track Bookings| ViewBookings[View Bookings]
    ViewBookings --> RateService{Rate?}
    RateService -->|Yes| SubmitRating[Submit Rating]
    SubmitRating --> Dashboard
    RateService -->|No| Dashboard

    Action -->|Chatbot| OpenChat[Chat with AI]
    OpenChat --> Dashboard

    Action -->|Logout| End([End])
```

4.6 Activity Diagrams

4.6.1 Customer Activity Diagram

Complete customer journey from registration through service booking, payment, and rating.

```
flowchart TD
    Start([Customer Opens App]) --> CheckAuth{Has Account?}
    CheckAuth -->|No| Register[Register as Customer]
    Register --> EnterDetails[Enter Email, Password, Name, Phone]
    EnterDetails --> CreateAuth[System Creates Firebase Auth Account]
    CreateAuth --> CreateProfile[System Creates Firestore Profile]
    CreateProfile --> Login
```

```
CheckAuth -->|Yes| Login[Login to System]
Login --> Dashboard[Customer Dashboard]

Dashboard --> Action{Choose Action}

Action -->|Manage Cars| AddCar[Add/Edit Car]
AddCar --> EnterCarDetails[Enter Car Details]
EnterCarDetails --> SaveCar[Save to Firestore]
SaveCar --> Dashboard

Action -->|Book Service| HasCar{Has Registered Car?}
HasCar -->|No| AddCar
HasCar -->|Yes| SelectCar[Select Car]
SelectCar --> SelectType[Select Maintenance Type]
SelectType --> BrowseServices[Browse Service Catalog]
BrowseServices --> SelectDateTime[Choose Date & Time]
SelectDateTime --> CheckSlot{Slot Available?}
CheckSlot -->|No| SelectDateTime
CheckSlot -->|Yes| EnterNotes[Enter Description]
EnterNotes --> ApplyOffer{Have Discount Code?}
ApplyOffer -->|Yes| EnterCode[Enter Offer Code]
EnterCode --> ValidateCode{Code Valid?}
ValidateCode -->|No| ShowError[Show Error]
ShowError --> EnterNotes
ValidateCode -->|Yes| ApplyDiscount[Apply Discount]
ApplyOffer -->|No| ConfirmBooking
ApplyDiscount --> ConfirmBooking[Confirm Booking]
ConfirmBooking --> SaveBooking[Save to Firestore]
SaveBooking --> NotifyTechs[Notify All Technicians]
NotifyTechs --> Dashboard

Action -->|Track Bookings| ViewBookings[View My Bookings]
ViewBookings --> SelectBooking[Select Booking]
SelectBooking --> ViewStatus[View Current Status & Details]
ViewStatus --> CheckComplete{Status = Completed & Paid?}
CheckComplete -->|Yes| RateService{Want to Rate?}
RateService -->|Yes| EnterRating[Enter Stars & Comment]
EnterRating --> SubmitRating[Submit Rating]
SubmitRating --> Dashboard
RateService -->|No| Dashboard
CheckComplete -->|No| Dashboard

Action -->|Download Invoice| SelectPaid[Select Paid Booking]
SelectPaid --> GeneratePDF[Generate PDF Invoice]
GeneratePDF --> DownloadPDF[Download to Device]
DownloadPDF --> Dashboard

Action -->|Chat with AI| OpenChatbot[Open Chatbot]
OpenChatbot --> LoadHistory[Load Conversation History]
LoadHistory --> TypeMessage[Type Message]
TypeMessage --> SendToAI[Send to Gemini AI]
SendToAI --> ReceiveResponse[Receive AI Response]
ReceiveResponse --> SaveConversation[Save to Firestore]
```

```
    SaveConversation --> ContinueChat{Continue Chatting?}
    ContinueChat -->|Yes| TypeMessage
    ContinueChat -->|No| Dashboard

    Action -->|Logout| End([End Session])
```

4.6.2 Technician Activity Diagram

Technician workflow from viewing available jobs through service execution and completion.

```
flowchart TD
    Start([Technician Login]) --> Dashboard[Technician Dashboard]

    Dashboard --> Action{Choose Action}

    Action -->|View Jobs| ViewAvailable[View Available Jobs]
    ViewAvailable --> JobList[Display Pending Bookings]
    JobList --> SelectJob{Select Job?}
    SelectJob -->|Yes| ViewDetails[View Job Details]
    ViewDetails --> Decision{Take Job?}
    Decision -->|No| JobList
    Decision -->|Yes| StartJob[Click Start Job]
    StartJob --> UpdateStatus1[Status: inProgress]
    UpdateStatus1 --> AssignSelf[Assign Self]
    AssignSelf --> RecordTime[Record startedAt]
    RecordTime --> NotifyCustomer1[Notify Customer]
    NotifyCustomer1 --> WorkOnJob[Perform Work]

    SelectJob -->|No| Dashboard

    Action -->|My Active Jobs| WorkOnJob
    WorkOnJob --> AddItems{Need Parts?}
    AddItems -->|Yes| SearchInventory[Search Inventory]
    SearchInventory --> CheckStock{Stock Available?}
    CheckStock -->|No| RequestRestock[Request Restock]
    RequestRestock --> WaitForStock[Wait]
    WaitForStock --> SearchInventory
    CheckStock -->|Yes| SelectItem[Select Item]
    SelectItem --> AddToBooking[Add to Service Items]
    AddToBooking --> UpdateInventory[Update Inventory]
    UpdateInventory --> DecrementStock[Decrement Stock]
    DecrementStock --> CheckThreshold{Below Threshold?}
    CheckThreshold -->|Yes| CreateAlert[Create Alert]
    CreateAlert --> NotifyAdmin[Notify Admin]
    NotifyAdmin --> AddItems
    CheckThreshold -->|No| AddItems
    AddItems -->|No| PerformWork[Perform Service]
    PerformWork --> EnterNotes[Enter Notes]
    EnterNotes --> Complete{Complete?}
    Complete -->|No| AddItems
```

```
    Complete -->|Yes| MarkComplete[Mark Complete]
    MarkComplete --> UpdateStatus2[Status: completedPendingPayment]
    UpdateStatus2 --> RecordCompletion[Record completedAt]
    RecordCompletion --> CalculateCost[Calculate Total Cost]
    CalculateCost --> NotifyCustomer2[Notify Customer]
    NotifyCustomer2 --> NotifyCashier[Notify Cashier]
    NotifyCashier --> Dashboard

    Action -->|View Inventory| ViewStock[View Stock Levels]
    ViewStock --> Dashboard

    Action -->|Logout| End([End Session])
```

4.6.3 Admin Activity Diagram

Administrative tasks including user management, invite codes, refund approval, and offers creation.

```
flowchart TD
    Start([Admin Login]) --> Dashboard[Admin Dashboard]
    Dashboard --> ViewNotifications[Load Notifications]
    ViewNotifications --> Action{Choose Action}

    Action -->|Manage Users| LoadUsers[Load All Users]
    LoadUsers --> SelectUser[Select User]
    SelectUser --> UserAction{Action?}
    UserAction -->|Toggle Status| ToggleStatus[Activate/Deactivate]
    ToggleStatus --> UpdateUser[Update Firestore]
    UpdateUser --> Dashboard
    UserAction -->|View Details| ShowDetails[Show Info]
    ShowDetails --> Dashboard

    Action -->|Invite Codes| CodeAction{Action?}
    CodeAction -->|Generate| SelectRole[Select Role]
    SelectRole --> EnterMaxUses[Enter Max Uses]
    EnterMaxUses --> GenerateCode[Generate Code]
    GenerateCode --> SaveCode[Save to Firestore]
    SaveCode --> DisplayCode[Display Code]
    DisplayCode --> Dashboard
    CodeAction -->|View| ViewCodes[View Codes]
    ViewCodes --> Dashboard

    Action -->|Refund Requests| ViewRefunds[View Pending Refunds]
    ViewRefunds --> SelectRefund[Select Refund]
    SelectRefund --> ReviewDetails[Review Details]
    ReviewDetails --> ApprovalDecision{Approve?}
    ApprovalDecision -->|Approve| ApproveRefund[Approve]
    ApproveRefund --> NotifyCashier1[Notify Cashier]
    NotifyCashier1 --> Dashboard
    ApprovalDecision -->|Reject| RejectRefund[Reject]
    RejectRefund --> NotifyCustomer[Notify Customer]
```

```
    NotifyCustomer --> Dashboard

    Action -->|Create Offers| EnterOfferDetails[Enter Details]
    EnterOfferDetails --> GenerateOfferCode[Generate Code]
    GenerateOfferCode --> SaveOffer[Save Offer]
    SaveOffer --> GetCustomers[Get All Customers]
    GetCustomers --> BatchNotify[Notify All]
    BatchNotify --> Dashboard

    Action -->|View Reports| SelectReport{Report Type?}
    SelectReport -->|Profit| DisplayProfit[Display Revenue]
    DisplayProfit --> Dashboard
    SelectReport -->|Performance| DisplayPerformance[Display Stats]
    DisplayPerformance --> Dashboard

    Action -->|Logout| End([End Session])
```

4.6.4 Cashier Activity Diagram

Cashier operations for payment processing, invoice generation, and refund initiation.

```
flowchart TD
    Start([Cashier Login]) --> Dashboard[Cashier Dashboard]

    Dashboard --> Action{Choose Action}

    Action -->|Process Payments| LoadPending[Load Pending Payments]
    LoadPending --> SelectBooking[Select Booking]
    SelectBooking --> LoadDetails[Load Details]
    LoadDetails --> ReviewCosts[Review Costs]
    ReviewCosts --> SelectMethod[Select Payment Method]
    SelectMethod --> PaymentType{Method?}

    PaymentType -->|Cash| ReceiveCash[Receive Cash]
    ReceiveCash --> ProcessSuccess[Process]

    PaymentType -->|Card| ProcessCard[Process Card]
    ProcessCard --> CardSuccess{Success?}
    CardSuccess -->|Yes| ProcessSuccess
    CardSuccess -->|No| PaymentFailed[Failed]
    PaymentFailed --> SelectMethod

    PaymentType -->|Digital| ProcessDigital[Process Digital]
    ProcessDigital --> ProcessSuccess

    ProcessSuccess --> UpdateBooking1[Update isPaid]
    UpdateBooking1 --> UpdateStatus[Update Status]
    UpdateStatus --> GenerateInvoice[Generate Invoice]
    GenerateInvoice --> SendToCustomer[Send Invoice]
    SendToCustomer --> Dashboard
```

```
    Action -->|Initiate Refund| LoadPaidBookings[Load Paid Bookings]
    LoadPaidBookings --> SelectForRefund[Select Booking]
    SelectForRefund --> EnterReason[Enter Reason]
    EnterReason --> CreateRefund[Create Refund]
    CreateRefund --> NotifyAdmin1[Notify Admin]
    NotifyAdmin1 --> Dashboard

    Action -->|View Reports| SelectDateRange[Select Date Range]
    SelectDateRange --> DisplayReport[Display Revenue]
    DisplayReport --> Dashboard

    Action -->|Logout| End([End Session])
```

4.6.5 User Authentication Activity Diagram

Complete authentication flow showing different paths for customer vs staff registration with invite code validation.

```
flowchart TD
    Start([User Opens App]) --> CheckSession{Has Session?}
    CheckSession -->|Yes| Redirect{Redirect by Role}
    Redirect -->|Customer| CustDash[Customer Dashboard]
    Redirect -->|Technician| TechDash[Technician Dashboard]
    Redirect -->|Admin| AdminDash[Admin Dashboard]
    Redirect -->|Cashier| CashDash[Cashier Dashboard]

    CheckSession -->|No| ShowAuth[Login/Register Screen]
    ShowAuth --> UserChoice{Action?}

    UserChoice -->|Login| EnterCreds[Enter Credentials]
    EnterCreds --> ValidateCreds[Validate]
    ValidateCreds --> CredsValid{Valid?}
    CredsValid -->|No| ShowLoginError[Show Error]
    ShowLoginError --> EnterCreds
    CredsValid -->|Yes| LoadProfile[Load Profile]
    LoadProfile --> CheckRole[Check Role]
    CheckRole --> Redirect

    UserChoice -->|Register| SelectRole{Select Role}

    SelectRole -->|Customer| DirectReg[Customer Registration]
    DirectReg --> EnterCustomerDetails[Enter Details]
    EnterCustomerDetails --> ValidateCustomer{Valid?}
    ValidateCustomer -->|No| ShowRegError1[Show Error]
    ShowRegError1 --> EnterCustomerDetails
    ValidateCustomer -->|Yes| CreateAuth1[Create Auth]
    CreateAuth1 --> CreateProfile1[Create Profile]
    CreateProfile1 --> CustDash
```

```
    SelectRole -->|Staff| StaffReg[Staff Registration]
    StaffReg --> ShowCodeField[Show Code Field]
    ShowCodeField --> EnterStaffDetails[Enter Details + Code]
    EnterStaffDetails --> ValidateStaff{Valid?}
    ValidateStaff -->|No| ShowRegError2[Show Error]
    ShowRegError2 --> EnterStaffDetails
    ValidateStaff -->|Yes| QueryCode[Query Code]
    QueryCode --> CodeExists{Exists?}
    CodeExists -->|No| ShowCodeError[Invalid Code]
    ShowCodeError --> EnterStaffDetails
    CodeExists -->|Yes| CheckCodeActive{Active?}
    CheckCodeActive -->|No| ShowCodeError
    CheckCodeActive -->|Yes| CheckCodeRole{Role Matches?}
    CheckCodeRole -->|No| ShowRoleMismatch[Wrong Role]
    ShowRoleMismatch --> EnterStaffDetails
    CheckCodeRole -->|Yes| CheckUsage{Within Max?}
    CheckUsage -->|No| ShowCodeExpired[Code Maxed]
    ShowCodeExpired --> EnterStaffDetails
    CheckUsage -->|Yes| CreateAuth2[Create Auth]
    CreateAuth2 --> CreateProfile2[Create Profile]
    CreateProfile2 --> IncrementCode[Increment usedCount]
    IncrementCode --> RedirectStaff{Redirect}
    RedirectStaff -->|Technician| TechDash
    RedirectStaff -->|Admin| AdminDash
    RedirectStaff -->|Cashier| CashDash
```

4.6.6 Data Collection Activity Diagram

System workflow for collecting, validating, and storing booking data with service selection.

```
flowchart TD
    Start([Customer Initiates Booking]) --> LoadCars[Load Cars]
    LoadCars --> CarsExist{Has Cars?}
    CarsExist -->|No| AddCarFlow[Add Car]
    AddCarFlow --> CollectCarData[Collect Car Data]
    CollectCarData --> ValidateCarData{Valid?}
    ValidateCarData -->|No| ShowCarError[Show Error]
    ShowCarError --> CollectCarData
    ValidateCarData -->|Yes| SaveCar[Save Car]
    SaveCar --> LoadCars

    CarsExist -->|Yes| DisplayCars[Display Cars]
    DisplayCars --> SelectCar[Select Car]
    SelectCar --> StoreCarId[Store carId]

    StoreCarId --> ShowTypes[Show Maintenance Types]
    ShowTypes --> SelectType[Select Type]
    SelectType --> StoreType[Store maintenanceType]

    StoreType --> LoadServices[Load Service Catalog]
```

```
    LoadServices --> DisplayServices[Display Services]
    DisplayServices --> BrowseServices{Select Services?}
    BrowseServices -->|Yes| SelectService[Select Service]
    SelectService --> AddToList[Add to List]
    AddToList --> BrowseServices
    BrowseServices -->|No| ReviewSelection{Has Items?}
    ReviewSelection -->|No| ShowWarning[Show Warning]
    ShowWarning --> DisplayServices
    ReviewSelection -->|Yes| StoreItems[Store serviceItems]

    StoreItems --> ShowCalendar[Show Calendar]
    ShowCalendar --> SelectDate[Select Date]
    SelectDate --> ShowTimeSlots[Show Time Slots]
    ShowTimeSlots --> SelectTime[Select Time]
    SelectTime --> CheckAvailability[Check Availability]
    CheckAvailability --> SlotAvailable{Available?}
    SlotAvailable -->|No| ShowAlternatives[Show Alternatives]
    ShowAlternatives --> SelectTime
    SlotAvailable -->|Yes| StoreDateTime[Store DateTime]

    StoreDateTime --> EnterDescription[Enter Description]
    EnterDescription --> StoreDescription[Store description]

    StoreDescription --> OfferPrompt{Enter Code?}
    OfferPrompt -->|Yes| EnterOfferCode[Enter Code]
    EnterOfferCode --> ValidateOffer[Validate Offer]
    ValidateOffer --> OfferValid{Valid?}
    OfferValid -->|No| ShowOfferError[Show Error]
    ShowOfferError --> OfferPrompt
    OfferValid -->|Yes| StoreOffer[Store Offer]
    StoreOffer --> DisplaySummary

    OfferPrompt -->|No| DisplaySummary[Display Summary]
    DisplaySummary --> Confirms{Confirm?}
    Confirms -->|No| Start
    Confirms -->|Yes| CreateBooking[Create Booking]
    CreateBooking --> SetStatus[Set status: pending]
    SetStatus --> SaveToFirestore[Save to Firestore]
    SaveToFirestore --> NotifyTechs[Notify Technicians]
    NotifyTechs --> ShowSuccess[Show Success]
    ShowSuccess --> End([End])
```

4.7 Sequence Diagrams

4.7.1 Customer System Browsing

```
sequenceDiagram
    participant C as Customer
    participant App as Flutter App
    participant FS as Firestore
```

```
    C->>App: Open Dashboard
    App->>FS: Query offers where isActive=true
    FS-->>App: Return Active Offers
    App->>FS: Query services where isActive=true
    FS-->>App: Return Service Catalog
    App-->>C: Display Offers & Services

    C->>App: View Offer Details
    App-->>C: Show Discount Code & Terms

    C->>App: Browse Service Catalog
    App-->>C: Display Services by Category
```

4.7.2 Technician System Browsing

```
sequenceDiagram
    participant T as Technician
    participant App as Flutter App
    participant FS as Firestore

    T->>App: Open Dashboard
    App->>FS: Query bookings where status=pending
    FS-->>App: Return Available Jobs
    App-->>T: Display Job List

    T->>App: Select Job
    App->>FS: getDoc(bookings, jobId)
    FS-->>App: Return Booking Details
    App->>FS: getDoc(cars, carId)
    FS-->>App: Return Car Details
    App->>FS: getDoc(users, customerId)
    FS-->>App: Return Customer Info
    App-->>T: Display Complete Job Details
```

4.7.3 Admin System Browsing

```
sequenceDiagram
    participant A as Admin
    participant App as Flutter App
    participant FS as Firestore

    A->>App: Open Dashboard
    par Load Dashboard Data
        App->>FS: Query all bookings
        App->>FS: Query all users
        App->>FS: Query low_stock_alerts where isResolved=false
        App->>FS: Query refunds where status=requested
    end
```

```
    FS-->>App: Return All Data Streams
    App-->>A: Display: Stats, Alerts, Pending Actions

    A->>App: View Reports
    App->>FS: Aggregate bookings by status & date
    FS-->>App: Return Aggregated Data
    App-->>A: Display Charts & Analytics
```

4.7.4 Cashier System Browsing

```
sequenceDiagram
    participant CS as Cashier
    participant App as Flutter App
    participant FS as Firestore

    CS->>App: Open Dashboard
    App->>FS: Query bookings where status=completedPendingPayment
    FS-->>App: Return Pending Payments
    App->>FS: Query refunds where status=approved
    FS-->>App: Return Approved Refunds
    App-->>CS: Display: Pending Payments, Approved Refunds

    CS->>App: Select Booking
    App->>FS: getDoc(bookings, bookingId)
    FS-->>App: Return Booking with Cost Breakdown
    App-->>CS: Show: Subtotal, Discount, Tax, Total
```

4.7.5 Customer Registration

```
sequenceDiagram
    participant U as User
    participant App as Registration Form
    participant Auth as Firebase Auth
    participant FS as Firestore

    U->>App: Enter: Email, Password, Name, Phone
    App->>App: Validate Input Format
    App->>Auth: createUserWithEmailAndPassword(email, password)
    Auth-->>App: Return UID
    App->>FS: Create document in users collection
    Note over FS: role=customer, isActive=true
    FS-->>App: User Created
    App->>Auth: signInWithEmailAndPassword(email, password)
    Auth-->>App: Sign In Success
    App-->>U: Redirect to Customer Dashboard
```

4.7.6 Technician Registration

```
sequenceDiagram
    participant U as User
    participant App as Registration Form
    participant FS as Firestore
    participant Auth as Firebase Auth

    U->>App: Enter: Email, Password, Name, Phone, Invite Code
    App->>FS: Query invite_codes where code=X
    FS-->>App: Return Invite Code Document
    App->>App: Validate: isActive=true, role=technician, usedCount>Auth:
createUserWithEmailAndPassword(email, password)
        Auth-->>App: Return UID
        App->>FS: Create user with role=technician
        App->>FS: Update invite_code: usedCount++, usedBy.add(userId)
        FS-->>App: Success
        App-->>U: Redirect to Technician Dashboard
    else Code Invalid
        App-->>U: Error: Invalid or Expired Code
    end
```

4.7.7 Admin Registration

```
sequenceDiagram
    participant U as User
    participant App as Registration Form
    participant FS as Firestore
    participant Auth as Firebase Auth

    U->>App: Enter: Email, Password, Name, Phone, Admin Code
    App->>FS: Query invite_codes where code=X
    FS-->>App: Return Invite Code
    App->>App: Validate: role=admin, isActive=true
    alt Valid Admin Code
        App->>Auth: createUserWithEmailAndPassword
        Auth-->>App: UID
        App->>FS: Create user with role=admin
        App->>FS: Update invite_code usage
        App-->>U: Redirect to Admin Dashboard
    else Invalid
        App-->>U: Error
    end
```

4.7.8 Cashier Registration

```
sequenceDiagram
    participant U as User
    participant App as Registration
    participant FS as Firestore
```

```
    participant Auth as Firebase Auth

    U->>App: Enter Details + Cashier Code
    App->>FS: Query invite_codes where code=X
    FS-->>App: Return Code (role=cashier)
    App->>App: Validate Code
    alt Valid
        App->>Auth: createUser
        App->>FS: Create user (role=cashier)
        App->>FS: Update code usage
        App-->>U: Redirect to Cashier Dashboard
    else Invalid
        App-->>U: Error
    end
```

4.7.9 Main Feature - Complete Booking Lifecycle

```
sequenceDiagram
    participant C as Customer
    participant App as App
    participant FS as Firestore
    participant T as Technician
    participant CS as Cashier

    Note over C,FS: Phase 1: Booking Creation
    C->>App: Create Booking
    App->>FS: addDoc(bookings, status=pending)
    FS-->>App: Booking Created
    App->>FS: Batch create notifications for all technicians
    FS-->>T: Notification: New Job Available
    FS-->>C: Confirmation

    Note over T,FS: Phase 2: Job Execution
    T->>App: Start Job
    App->>FS: Update: status=inProgress, assignedTechnicians=[techId]
    FS-->>C: Notification: Job Started
    T->>App: Add Service Items
    App->>FS: Update serviceItems[], create inventory_transaction
    T->>App: Complete Job
    App->>FS: Update: status=completedPendingPayment, completedAt=now
    FS-->>CS: Notification: Payment Pending
    FS-->>C: Notification: Service Complete

    Note over CS,FS: Phase 3: Payment
    CS->>App: Process Payment
    App->>FS: Update: isPaid=true, status=completed
    App->>FS: Create invoice document
    FS-->>C: Invoice + Notification

    Note over C,FS: Phase 4: Rating
    C->>App: Submit Rating
```

```
    App->>FS: Update: rating, ratingComment, ratedAt
    FS-->>T: Notification: Received Rating
```

4.7.10 System Monitoring

```
sequenceDiagram
    participant A as Admin
    participant App as Admin Panel
    participant FS as Firestore
    participant Alert as Alert System

    Note over A,FS: Real-time Monitoring
    A->>App: Open Dashboard
    loop Real-time Streams
        FS->>App: Stream: New Bookings
        FS->>App: Stream: Payment Updates
        FS->>App: Stream: Low Stock Alerts
        FS->>App: Stream: Refund Requests
    end
    App-->>A: Display All Notifications

    Note over Alert,FS: Inventory Alert
    FS->>Alert: Trigger: currentStock < threshold
    Alert->>FS: Create low_stock_alert
    Alert->>FS: Create user_notification for admin
    FS->>App: Real-time Update
    App-->>A: Alert: Low Stock on Item X

    Note over A,FS: Performance Monitoring
    A->>App: View Performance Report
    App->>FS: Aggregate technician stats
    App->>FS: Aggregate booking stats
    FS-->>App: Return Metrics
    App-->>A: Display: Completed Jobs, Average Rating, Revenue
```

4.8 State Diagrams

4.8.1 Customer State Diagram

```
stateDiagram-v2
    [*] --> NotRegistered
    NotRegistered --> Registered : Complete Registration
    Registered --> Active : Add First Car
    Active --> Booking : Create Booking
    Booking --> TrackingService : Technician Starts Job
    TrackingService --> AwaitingPayment : Service Completed
    AwaitingPayment --> PaymentComplete : Payment Processed
    PaymentComplete --> RatingService : Click Rate Service
```

```
    RatingService --> Active : Submit Rating
    PaymentComplete --> Active : Skip Rating
    Active --> Booking : Book Another Service
    Active --> [*] : Account Deactivated
```

4.8.2 Technician State Diagram

```
stateDiagram-v2
    [*] --> Idle
    Idle --> BrowsingJobs : View Available Jobs
    BrowsingJobs --> Idle : No Jobs Selected
    BrowsingJobs --> Working : Start Job
    Working --> AddingItems : Search Inventory
    AddingItems --> Working : Items Added
    Working --> Completing : Mark Complete
    Completing --> Idle : Job Submitted
    Idle --> [*] : Logout
```

4.8.3 Admin State Diagram

```
stateDiagram-v2
    [*] --> Monitoring
    Monitoring --> UserManagement : Manage Users
    UserManagement --> Monitoring : Done
    Monitoring --> CodeGeneration : Generate Invite Codes
    CodeGeneration --> Monitoring : Code Created
    Monitoring --> RefundApproval : Review Refund Request
    RefundApproval --> ApprovingRefund : Approve
    RefundApproval --> RejectingRefund : Reject
    ApprovingRefund --> Monitoring : Cashier Notified
    RejectingRefund --> Monitoring : Customer Notified
    Monitoring --> CreatingOffer : Create Offer
    CreatingOffer --> Monitoring : Customers Notified
    Monitoring --> ViewingReports : View Reports
    ViewingReports --> Monitoring : Done
    Monitoring --> [*] : Logout
```

4.8.4 Cashier State Diagram

```
stateDiagram-v2
    [*] --> Idle
    Idle --> ReviewingPayments : View Pending Payments
    ReviewingPayments --> ProcessingPayment : Select Booking
    ProcessingPayment --> SelectingMethod : Review Costs
    SelectingMethod --> ExecutingPayment : Choose Method
    ExecutingPayment --> PaymentSuccess : Payment OK
```

```
    ExecutingPayment --> PaymentFailed : Payment Failed
    PaymentFailed --> SelectingMethod : Retry
    PaymentSuccess --> GeneratingInvoice : Create Invoice
    GeneratingInvoice --> Idle : Invoice Sent
    Idle --> InitiatingRefund : Customer Requests Refund
    InitiatingRefund --> RefundRequested : Submit to Admin
    RefundRequested --> Idle : Admin Notified
    Idle --> ViewingReports : View Reports
    ViewingReports --> Idle : Done
    Idle --> [*] : Logout
```

4.8.5 Booking Lifecycle State Diagram

```
stateDiagram-v2
    [*] --> Pending : Customer Creates Booking
    Pending --> InProgress : Technician Starts Job
    Pending --> Cancelled : Customer/Admin Cancels
    InProgress --> CompletedPendingPayment : Technician Completes
    CompletedPendingPayment --> Completed : Cashier Processes Payment
    Completed --> [*] : Workflow Ends
    Cancelled --> [*] : Workflow Ends

    note right of Pending
        Visible to all technicians
        Notifications sent
    end note

    note right of InProgress
        Technician assigned
        Service items being added
        Inventory updated
    end note

    note right of CompletedPendingPayment
        Total cost calculated
        Awaiting payment
    end note

    note right of Completed
        Payment received
        Invoice generated
        Ready for rating
    end note
```

4.8.6 Refund Request State Diagram

```
stateDiagram-v2
    [*] --> Requested : Cashier Initiates Refund
    Requested --> Approved : Admin Approves
```

```
    Requested --> Rejected : Admin Rejects
    Approved --> Processed : Cashier Completes Refund
    Processed --> [*] : Refund Complete
    Rejected --> [*] : Request Denied

    note right of Requested
        requestedBy: cashierId
        Reason & amount specified
        Admin notified
    end note

    note right of Approved
        approvedBy: adminId
        Ready for processing
    end note

    note right of Processed
        Payment reversed
        Customer notified
    end note
```

Summary

This document contains complete UML diagrams for the Fix-Hub Car Maintenance Management System:

- **Use Case Tables**: 15+ detailed use cases covering all 4 roles
- **Activity Diagrams**: 6 diagrams (Customer, Technician, Admin, Cashier, Authentication, Data Collection)
- **Sequence Diagrams**: 10 diagrams (4 browsing, 4 registration, booking lifecycle, monitoring)
- **State Diagrams**: 6 diagrams (Customer, Technician, Admin, Cashier, Booking, Refund)

All diagrams use Mermaid syntax for compatibility with VS Code, GitHub, and documentation tools.